Step 1 One-Pager — Reinforcement Learning Basics

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Problem. Everything later in this plan — neural equilibrium finding (Step 5), the opponent-model actuator (Steps 7-8), multi-agent learning (Step 9) — is assembled out of value networks, policy gradients and experience replay. Step 1 is the foundation layer: how an agent improves a policy from reward alone, in the fully observable single-agent setting, before that picture is complicated by a second, hidden, adversarial player. It feeds Contribution #1 indirectly (the architecture patterns are reused by the adaptive opponent model) and directly supplies the implementation vocabulary every later step assumes.

Approach. One algorithm from each family, written from scratch in PyTorch: **DQN** (value-based, off-policy) on `CartPole-v1` — Q-network `[4->128->128->2]`, circular replay buffer, frozen target network synced every 5 episodes — and **PPO** (policy-gradient, on-policy) on `LunarLander-v3` — separate actor `[8->128->128->4]` and critic, GAE (`lambda=0.95`) by reverse sweep, clipped surrogate at `eps=0.2`. Each was then benchmarked against its **Stable-Baselines3** counterpart at matched core hyperparameters and matched step budgets (DQN 750K, PPO 500K), scored on best rolling-100 average so that early stopping cannot flatter the result. **All numbers below are measured.**

Key results (measured).

- *Both targets met.* DQN solves CartPole at **episode 1011**, 100-episode average **477.5** (target 475); PPO solves LunarLander at **264,192 steps** at **202.2** (target 200) — 236K steps inside its budget.
- *The from-scratch versions beat the SB3 defaults on these tasks.* Best rolling-100: DQN **477.5 vs 293.9**, PPO **203.6 vs 131.2**. The cause is algorithmic, not unfair tuning: episode-based epsilon decay against SB3's step-based one, an adaptive target sync (5 episodes is ~100 steps early and ~2500 once solved) against a fixed 1000 steps, and early stopping against running the full budget.
- ***`eps_min` was the decisive DQN knob.*** At 0.01 the run peaked at **479.1** (episode 810) and decayed to **302.6**; 0.001 plus best-model checkpointing turned a degrading policy into a stable one.
- *Capacity, not budget, was PPO's bottleneck.* `[64,64]` peaked at **179.9** and never crossed 200; `[128,128]` cleared it. Advantage normalisation proved non-negotiable on rewards spanning -500 to +300, and without the 0.01 entropy bonus the policy collapsed to hovering in place.
- *Honest read on the comparison.* SB3's defaults are tuned for robustness across environments, not for classic control; its PPO was still climbing at 500K and would likely reach target given 1-2M steps. The claim is a task-specific advantage, not a better algorithm.

Thesis connection. The 300-500-line stack (against SB3's ~10K) is the deliberate price paid for surgical modifiability: injecting belief-state observations into an actor-critic is a deep subclassing exercise in SB3 and a local edit here, which is exactly what the exploitation work in Steps 7-8 needs. The same actor-critic pair returns as MAPPO/MADDPG in Step 9, and the value-network machinery returns as Deep CFR's advantage network in Step 5.

Open questions. The MDP assumption that makes all of this work — a fully observable state — is precisely what poker breaks: over an information set, DQN's `max` and PPO's per-state policy are both ill-posed, which is why Step 2 changes tool rather than scale. Open: how much of the single-agent stability toolkit (target networks, trust regions, advantage normalisation) survives when the environment is itself a learning agent (deferred to Step 9's non-stationarity), and whether the sample-efficiency edge measured here is a real property or an artifact of two small, well-conditioned control tasks.